

React.js

1. What are props in React?

- ❖ Props means Properties in React.
- ❖ To Pass Data Parent Component To Child Component.
- ❖ Props Are Read-Only.
- ❖ Props Are immutable means Can't modify the value.
- ❖ You **cannot modify props** inside the child component.
- ❖ You **can modify props** inside the Parent component.
- ❖ Controlled by the parent – Any changes must come from the parent.
- ❖ Props can be any type — strings, numbers, arrays, objects, functions, JSX elements, etc.
- ❖ Components re-render automatically when props change.
- ❖ Dynamic Rendering.

2. What is the purpose of Destructuring Props?

- ❖ Avoids repeating **props** multiple times.
- ❖ Destructured variables show up clearly.
- ❖ Easier Maintenance.
- ❖ Better Autocomplete & Type Safety
- ❖ Improved Readability
- ❖ Cleaner Code

3. What is useState and purpose?

- ❖ **useState** is a Hook in React
- ❖ add state to functional components.
- ❖ it returns us two things:
 - 1.The current value of the state.
 - 2.A function to update that state value.
- ❖ React re-renders the component with updated state value.
- ❖ Not available in class components

4. What is useEffect and purpose?

- ❖ useEffect is used to handle side effects in functional components.
- ❖ Fetches data from API
- ❖ Updates the DOM manually
- ❖ Subscribes to event
- ❖ Sets a timer / interval
- ❖ Logs, validates input
- ❖ Navigates after login
- ❖ Runs after render
- ❖ Runs again only when dependencies change
- ❖ Can clean up when component unmounts

5. What is jsx?

- ❖ JSX stands for JavaScript XML.
- ❖ JSX allows you to write HTML-like elements directly in JavaScript code
- ❖ You can use curly braces {} in JSX to insert JavaScript Dynamic Values.
- ❖ JSX elements must return only one parent/root element.
- ❖ JSX looks like HTML but it's not.
- ❖ Browsers don't understand JSX.
- ❖ JSX makes it easier to create and manage React components
- ❖ Jsx is better readability and maintainability.

6. How do you create a functional component in React?

Functional components are JavaScript functions that return JSX.

They are simple, reusable, and support React Hooks.

Easier to read, test, and reuse than class components.

They do not use this keyword.

Support React Props.

Shortcuts for create functional components like

rafc → **reactArrowFunctionComponent**

rafce → **reactArrowFunctionExportComponent**

rafcp → **reactArrowFunctionExportComponentwithpropTypes**

7. What is the virtual DOM?

- ❖ React converts your JSX code into a **Virtual DOM (JavaScript object)**.
- ❖ React takes this Virtual DOM and creates the actual **Real DOM** in the browser.
- ❖ When state or props change, React creates a **new Virtual DOM**.
- ❖ React compares the **new Virtual DOM** with the **Old Virtual DOM** to find what has changed.
- ❖ This process is called "**diffing**".
- ❖ Update Only Changed Part in the real DOM not the entire page
- ❖ React calculates the **minimum number of changes** needed and updates only those parts in the **real DOM**, instead of re-rendering the entire page.
- ❖ This improves performance and user experience.

8. Real DOM vs Virtual DOM

Real DOM:

- Update speed slow
- Re-renders entire tree or Entire Dom
- Manipulation Directly by the browser
- Memory usage High
- Hard performance

Virtual DOM:

- ❖ Fast UI updates
- ❖ Updates only what's changed
- ❖ Managed by React
- ❖ Memory usage low
- ❖ Smooth performance
- ❖ Smart component re-rendering

9. How does React handle form submissions?

- ❖ React handles form submission via **onSubmit** event handler.
- ❖ Inputs are controlled via **useState**.
- ❖ Values are updated via **onChange** event handlers.
- ❖ **event.preventDefault()** prevents the default page reload.
- ❖ Use **useState()** to store and update input values.
- ❖ You can add client-side validation before submitting.
- ❖ Display error messages using conditional rendering.
- ❖ Input values come from the component's state, not the DOM.
- ❖ You can validate and send data using **fetch** or **axios**.
- ❖ Event Handling
- ❖ Controlled Components

10. What are Controlled Components in React?

- ❖ Controlled Components are input elements.(like `<input>`, `<textarea>`, `<select>`).
- ❖ Inputs are controlled via `useState`.
- ❖ Any change to the input is handled by React through state and event handlers.
- ❖ Bind form input value to state using `useState` in React.
- ❖ simplest and most common pattern in React for controlled form inputs.

11. How do you pass data from child to parent? (lifting state up process)

To pass data **from child to parent**, you use a **callback function**.

- The parent component passes a function as a prop (`sendDataToParent`) to the child component.
- Example: `<Child sendDataToParent={handleDataFromChild} />`
- Here is the Callback function “`handleDataFromChild()`”.
- The child component calls that function and passes data to the parent component via the `onClick` event.
- The parent component receives the data in its function and updates its state.

12. What is prop drilling?

- ❖ Data Passed Through Multiple Levels.
- ❖ Props are passed down from parent to deeply nested child components.
- ❖ Components that don't need the data still receive and forward props.
- ❖ Tight Coupling of Components.
- ❖ Reduced Maintainability.
- ❖ Not practical for large applications with deeply nested components.
- ❖ Solution: Context API or State Management Libraries

13. How can you avoid prop drilling?

- ❖ To avoid Prop Drilling, I use the React Context API.
- ❖ It works like a global state.
- ❖ We can directly access the data without passing props from parent to child components using `useContext()`.
- ❖ You share data globally without passing it manually through every component.
- ❖ keeps the code clean, simple, and maintainable.

14. How do you handle errors in React?

In React, error handling can be approached in several ways depending on where the error occurs.

1. Using Error Boundaries for rendering errors
2. Try-Catch Blocks for event handlers
3. Try-Catch Blocks for sync/async functions
4. State + conditional UI for User input/form validation
5. `componentDidCatch` for Logging/reporting

15. What is the difference between functional and class components?

Functional components:

- Functional components are cleaner in syntax and more straightforward.
- Functional components **can use hooks** (`useState`, `useEffect`, `useContext`, etc.).
- **Functional components are preferred** in modern React.
 - Easier to test and debug
 - Encouraged by React team
 - Can use hooks
 - more straightforward and readable

With **hooks**, logic can be reused in functional components via **custom hooks**.

Class components

- ❖ Class Components are more complex in syntax and require more setup.
- ❖ Class components **cannot** use hooks.
- ❖ In class components, code reuse typically requires HOCs or render props, which are more complex.
- ❖ `componentDidMount()` is the **Mount lifecycle method** in **React class components**.
- ❖ `componentDidUpdate()` is an **Update lifecycle method** in **React class components**.
- ❖ `componentWillUnmount()` is an **Unmount lifecycle method** in **React class components**.

If you're using **functional components**, `componentDidMount()` is **replaced by hooks**.

16. What is the role of defaultProps?

- ❖ Used to provide default values for props if they are not passed by the parent component.
- ❖ It works only when the prop is `undefined`
- ❖ It does not work when the prop is `null`.
- ❖ It can be used in both class components and function components.
- ❖ If the **parent component passes a prop**, that value is used.
- ❖ If the **parent does not pass the prop**, the **default value is used** from `defaultProps`.

17. How do you debug React apps?

"I combine browser tools, React Developer Tools, logging, breakpoints, and testing strategies to efficiently debug and maintain React applications."
Check the network tab for API calls.

Common Debug Scenarios I Handle

- ❖ Props not updating
- ❖ State not updating
- ❖ Component not rendering
- ❖ Data not displaying

18. What are keys in React lists?

1. Keys in React are used to uniquely identify list items.
2. So React can efficiently update and manage changes (such as adding, removing, or reordering items.) in the DOM.
3. DOM Speed faster
4. Avoid unnecessary re-renders.

19. What is the purpose of React.Fragment?

We use `React.Fragment` to group elements without creating an extra `div` in the DOM.

Using `React.Fragment` prevents adding unnecessary `<div>`.

HTML structure clean and lightweight.

No extra HTML tags

Clean and simple structure

Better performance

Easier styling

Short syntax available

The browser can render and update the UI more efficiently.

You can use `<>...</>` instead of writing

`<React.Fragment>...</React.Fragment>` — shorter and cleaner.

Without React.Fragment (using extra <div>)

```
function Example() {
  return (
    <div>
      <h1>Hello</h1>
      <p>Welcome to React!</p>
    </div>
  );
}
```

With React.Fragment:

```
function Example() {
  return (
    <React.Fragment>
      <h1>Hello</h1>
      <p>Welcome to React!</p>
    </React.Fragment>
  );
}
```

20. How do you manage Global State in React?

- ❖ Global state means **data shared by multiple components**
- ❖ To share data easily between components.
- ❖ To avoid prop drilling.
- ❖ When **many components need the same data**
- ❖ Tools to Manage Global State.
- ❖ You can use Context API, Redux, Zustand, or Recoil.
- ❖ Context API: Small / medium apps,.
- ❖ Redux Toolkit: Large apps
- ❖ Zustand: Small / quick projects, Lightweight and easy to use.
- ❖ Recoil : Modern alternative
- ❖ I Used Context API Works.
- ❖ Best for small to medium apps.
- ❖ Uses **createContext**, **Provider**, and **useContext**.
- ❖ Use **createContext()** → Create a global context.
- ❖ Use **Provider** → Wrap your app and share data.
- ❖ Use **useContext()** → Access data anywhere.
- ❖ Use **useMemo** and **React.memo** for Preventing unnecessary re-renders.
- ❖ **Splitting large Contexts**: Use smaller Contexts so only the needed parts re-render.
- ❖ **Code splitting**: Load only the parts of your app that are needed (lazy loading).

Advantages of Global State

Easy data sharing between components.
 No prop drilling needed.
 Central control over app data.
 Clean and organized code.
 Better performance and faster UI updates.
 Good for authentication, theme, language, etc.
 Easier debugging and scaling

21. What is Context API and how does it work?

- ❖ To **share data globally** across components.
- ❖ **Send data directly** to any component **without passing props manually**.
- ❖ To **avoid prop drilling**.
- ❖ To share common data like user info, theme, or language in multiple components.
- ❖ You create context with `createContext()` :What data you want to share.
- ❖ **Provide Data** → `Context.Provider`:The components and gives access to the data.
- ❖ `useContext()`: Any component can read or update that data or access the data.
- ❖ Best for **small to medium apps**.
- ❖ Easy to implement into react.
- ❖ Not use **very large apps** with complex logic.
- ❖ User authentication (logged-in user info)
- ❖ Theme settings (light/dark mode).
- ❖ Language or localization settings.
- ❖ Shopping cart or application-wide configuration.

Advantages of Context API

No more prop drilling
 Easy to use and built-in
 code clean and readable.
 Centralized way to manage shared data.

22.How do you optimize React app performance?

- ❖ Use memoization, code splitting, lazy loading, and virtualization.
- ❖ Use Memoization
- ❖ Use `React.memo` to prevent re-rendering
- ❖ Use `useMemo` to cache expensive calculations.
- ❖ Use `useCallback` to memoize functions passed as props.
- ❖ **Code splitting**:Code splitting means loading your app in small pieces so the page loads faster and only loads what is needed.
- ❖ Split code using `dynamic import()` statements.
- ❖ **Lazy Loading**:Load components or pages **only when needed(Saves time and memory)**.
- ❖ **Virtualization**:large lists or tables, render only what's visible on the screen.
- ❖ **Optimize Images and Assets**:Load large images lazily.

Advantages of Optimization:

Faster loading and smoother UI
Less memory usage
Better user experience
Efficient rendering — React updates only what's needed

23.What is memoization in React?

- ❖ Memoization in React is an **Improvement** technique.
- ❖ To avoid unnecessary recalculations.
- ❖ To avoid unnecessary re-renderings by reusing previous results when state or props haven't changed.
- ❖ improving performance.

3 Ways to Use Memoization in React

React.memo(): Prevents functional components from re-rendering if their props or state haven't changed.

useMemo(): Caches a computed value between renders.

useCallback():Prevents the function from being re-created unless its dependencies change.

When to Use Memoization

- ❖ components are heavy.
- ❖ You face performance issues due to frequent re-renders.

24. What is useCallback and when to use it?

useCallback is used to stop unnecessary creation of functions and avoid unwanted child re-renders.

Without **useCallback**:

- ❖ Every time your component renders, a new function is created.
- ❖ If you send this function to a child component (as props), the child thinks:
“Oh! A new function came, I must re-render.”
- ❖ This causes unnecessary re-renders → the app becomes slow.

With **useCallback**:

- ❖ useCallback **remembers** the function.
- ❖ It does NOT create a new function on every render.
- ❖ Child component does NOT re-render unnecessarily.

25. What are React performance tools?

React offers several performance tools:

React DevTools Profiler – To find slow components.

Components tab – To track props/state changes.

Chrome Performance tab – To see overall browser performance.

Lighthouse – To check loading speed and performance score.

Why Did You Render – To detect unnecessary re-renders.

26. What are common security issues in React?

- ❖ Cross-Site Scripting (XSS)
- ❖ Insecure APIs / Data Exposure
- ❖ Improper Authentication and Authorization
- ❖ Exposing Environment Variables
- ❖ Third-party Library Vulnerabilities
- ❖ Clickjacking
- ❖ Insecure Local Storage

27. How do you structure a large-scale React app?

Organize by feature folders or domains

(e.g., /users,

/orders, /components, /auth, /students, /hooks, /layouts, /pages/routes).

Scalability

Reusability

Separation of concerns

UI in **components/**

Business logic in **features/**

Network logic in **services/**

Routing in **routes/**

Testing in **test/**

Form Handling **form/**

API Calls in **service/**

State Management in **hooks/**

28. How do you handle authentication in React?

- ❖ I handle authentication in React using JWT.
- ❖ I send login details to the backend, receive a JWT token, store it in **localStorage** or **sessionStorage** or **React state / Context**
- ❖ We attach the token every API call.
- ❖ We create a Protected Route so only logged-in users can see certain pages like Dashboard, Profile.
- ❖ On logout, I remove the token.

Advantages

Token stored safely

Interceptor adds token automatically

- No need to repeat code
- Secure pages using Protected Routes
- Easy to manage user roles (admin/user)
- Faster API calls
- No prop drilling
- Works with any backend
- Supports token refresh easily

29. What is HOC (Higher Order Component)?

A Higher Order Component is a function that takes a component and returns a new component with extra features.

It adds extra features. Like

- adding loading
- adding authentication check
- adding logging
- adding common logic
- Logging user actions
- Authentication check
- Adding loading spinners
- Showing error boundaries
- Fetching data

Without changing the original component.

Without HOC: We need to add a "loading spinner" in many components → repeated code.

With HOC: We create **one HOC** that adds a loading spinner to any component.

30. Why use HOC instead of copying logic?

- ❖ To avoid code duplication.
- ❖ Reuse common logic
- ❖ Reduce repeated code
- ❖ Add extra behavior to many components
- ❖ Makes components cleaner
- ❖ To separate concerns and improve maintainability.

31. Difference between HOC and Render Props?

- ❖ **HOC:** Adding an extra layer around a component.
Gives **extra features** without changing original code
You use it by **calling a function**
Example: `withAuth(Component)`
- ❖ **Render Props:** Giving data through a function so you choose what to show
Uses a **function inside props**
You choose **what to render**
Passes **data through that function**

32. What is a custom hook and why would you use one?

A custom hook is a reusable function that contains React logic. We use it to reuse code, avoid duplication, and keep components clean and simple.

I use custom hooks to avoid duplication, simplify components, and organize logic better.

Must start with `use` (e.g.).

`useFetch` → API calling logic reuse

`useLocalStorage` → save data in localStorage easily

`useAuth` → check login status

`useDebounce` → delay input calls

`useCounter` → reusable counter logic

When you repeat the same logic in multiple components.

etc.).

When organizing code into smaller, reusable, testable parts.

Common Use Cases

Data fetching, Auth status, Form handling, Media queries, LocalStorage logic

33. How do you handle pagination in React?

You can implement pagination either on the client side or server side, depending on your use case.

Client-Side Pagination:

Fetch all the data initially.

Store data in state (`items`).

Store current page in state (`currentPage`).

Slice the data

Render them and show page buttons.

Small datasets

When performance is not an issue

Server-Side Pagination

Best for large datasets.

The backend sends only the data for the requested page.

Send the page number and page size in API request.

Backend returns data and metadata (e.g., total count).

React stores data and renders the current page.

34. Explain React Refs and useRef hook.

In React, Refs (short for References) are used to directly access and interact with DOM elements.

Continue mutable values across renders without Generating re-renders.

What is a Ref in React?

- ❖ A ref (short for "reference")
- ❖ **Directly access and interact with DOM elements**, such as inputs, buttons, or divs.
- ❖ **Store mutable values** that should *persist across renders without* causing re-renders.
- ❖ **mutable values**: Values that can **change** (update anytime). (Timer ID, Previous input value, Scroll position)
- ❖ **Persist across renders**: The value should **stay the same** even if the component renders again.
- ❖ **Example**: Component re-renders → but your stored value remains safe.
- ❖ **Without causing re-renders**: When this value changes, **the UI should NOT re-render**.
- ❖ **Example**: If you store something in `useState`, React re-renders the component.
But if you store it in a **ref**, React does **not** re-render.

What is a `useRef` Hook?

- ❖ Storing Mutable Values (e.g., timer ID, previous value)
- ❖ Some values need to be saved but should not update the UI.
- ❖ `useRef` stores those values safely without re-rendering the component.

35. What is useMemo and when to use it?

- ❖ `useMemo` memoizes the returned value of a function.
- ❖ It's used to optimize performance by preventing expensive calculations from running on every render, and it only recalculates when its dependency values change.

- ❖ Common use cases include sorting, filtering, and computing derived data, Complex or expensive calculations, arrays passed as props.

36. How do you handle side effects in React?

- ❖ Fetching data from an API
- ❖ Updating the DOM manually
- ❖ Setting up subscriptions or timers
- ❖ Logging
- ❖ Interacting with browser APIs (like `localStorage`)
- ❖ The Main Tool: `useEffect` Hook

37. How does React handle component lifecycle?

1. Lifecycle in Functional Components (Using Hooks)

Functional Components with Hooks: Modern standard for all new React apps.

useState: Local state management

useEffect: Side effects (fetch, DOM manipulation, etc.)

useRef: Continuous mutable values across renders

useMemo: Memoize expensive computations

useCallback: Memoize functions to avoid unnecessary renders

2. Lifecycle in Class Components

Class Components: Still supported, but not recommended for new development.

componentDidMount: Called after the component is mounted — good for API calls, subscriptions

componentDidUpdate: Acts after updates — good for side effects.

componentWillUnmount: Cleanup logic like unsubscribing or aborting fetch calls.

38. What is the difference between `useEffect` and `componentDidMount`?

`componentDidMount`

- ❖ Class Components only
- ❖ Called after the first render (once)
- ❖ Part of class lifecycle methods
- ❖ Cleanup Support Use `componentWillUnmount` separately
- ❖ Only one `componentDidMount` allowed per class

`useEffect`

- ❖ Functional Components only
- ❖ You can manage side effects in one place.
- ❖ After render (on mount or every update, depending on dependencies)
- ❖ Hook — more flexible and declarative
- ❖ Cleanup is handled inside `useEffect` return function
- ❖ Multiple `useEffects` allowed per function component

39. How do you implement code splitting in React?

Code splitting = faster apps by loading only what's needed.

Without Code Splitting: All pages and components load at once — **X**slow for big apps.

With Code Splitting: Only the home page loads first. Other pages load **only when visited** — **✓**faster!

Why Code Splitting?

Slower initial page load

Poor performance on slower networks or devices

Code splitting helps us?

Loading only the necessary code for the current view

Faster initial load.

Reducing initial bundle size

Improving load time and user experience

Better scalability for large apps.

Improve performance on mobile networks.

React gives you a simple way using:

React.lazy() :dynamically loads the component.

and **Suspense**: shows a fallback UI while loading.

40. How do you lazy load components in React?

React provides two built-in tools:

1. **React.lazy()**

2. **<Suspense>**

- ❖ Use **Suspense** around any lazy-loaded component.
- ❖ Lazy loading only works for default exports.
- ❖ Can be used with components, routes, modals, etc.

What is **React.lazy()**?

React.lazy() is a way to **load a component only when it is actually needed**, not at the beginning when the app starts.

Without React.lazy(): All 10 pages are loaded at the start — even if the user only visits 1 page.

With React.lazy():

- ❖ Only the first page loads.
- ❖ The other pages load **only when the user clicks on them**.
- ❖ Your app becomes **faster** and **lighter** at the beginning.
- ❖ Users don't have to wait for unused parts to load.

What is **<Suspense>** in React?

When you use **React.lazy()** to load a component later, it might take a few seconds to load. During that time, React needs to show something on the screen (like a message or spinner).

Think of it like this:

- ❖ Imagine you're waiting for a file to download.
- ❖ While it's downloading, you see a message: "Loading...".
- ❖ After it's ready, the real content is shown.

<Suspense> = the part that handles **"show while waiting"**

fallback={<div>Loading...</div>} = the **message** or **spinner** shown while waiting

41. What is React Router and how is it used?

React Router is a library for React that helps you build single-page applications (SPAs) with navigation between different views (pages) — without reloading the browser.

It allows your app to show different components when the URL changes, just like traditional websites, but **without refreshing the page**.

Why Use React Router?

- Create multiple "pages" in your React app
- Navigate between pages using links
- Control which component shows based on the URL

BrowserRouter: Sets up routing

Route: Maps a path to a component

Routes: Wraps all the routes

Link: Used to navigate between routes